# ADR 0009 — Abort and rollback (reversibility, undo_action)

**Status**: Proposed for v1 contract; rollback execution deferred to v2 (per RONIN_SYNC.txt Tier 3 #15) **Date**: 2026-06-20 **Author**: Marv **Related**: ADR 0001, ADR 0002, docs/DEFERRED.md

---

## Context

The agent loop (per ADR 0001) can hit a failure or an abort condition AFTER some tools have already fired. For example:

- A 3-step plan: `clipboard.read` → `notion.create_task` → `gmail.send`
- `notion.create_task` succeeds (task created in Notion)
- `gmail.send` fails (rate limit exceeded, won't recover by retry within budget)

What should the agent do? Three rough categories:

1. **Bail and leave state inconsistent** — task created, email not sent, user has no idea
2. **Roll back the successful step** — delete the just-created Notion task
3. **Commit nothing until everything succeeds** — buffer all side effects, only fire on full success

Option 1 is what bad agents do. Option 3 is two-phase commit, which doesn't compose with third-party APIs (you can't ask Notion to "prepare a task and commit later"). Option 2 is what mature systems do — every successful step exposes an inverse, and abort triggers reverse-order undo.

But tools differ in whether reverse is possible:

- `notion.create_task` — reversible (delete the task by ID)
- `calendar.create_event` — reversible (delete the event)
- `clipboard.read` — read-only, no side effect to reverse
- `gmail.send` — **irreversible** (you can't unsend an email)

For irreversible tools, the right pattern is verification BEFORE the action fires, not undo AFTER.

The locked decision from RONIN_SYNC.txt #15: "each tool adapter declares `reversible: bool` + `undo_action()`." The contract part is in v1; the execution (real rollback semantics with real adapters) is deferred to v2 when real adapters land.

---

## Decision

### Contract (v1)

Every tool adapter declares two attributes at registration time:

```
class ToolAdapter:
    name: str                       # e.g., "notion.create_task"
    reversible: bool                # True if undo_action exists
    def execute(self, args) -> ToolResult: ...
    def undo_action(self, action_log_entry) -> UndoResult: ...   # required iff reversible=True
```

The agent core invokes adapters through the validator (per ADR 0002) and uses the `reversible` flag in two places:

1. **Verification gate (before execution)**: if `reversible == False`, the agent emits a clarification-style prompt asking the user to confirm before the tool fires. v1 fake adapters in the eval harness are all reversible, so the gate never triggers during baseline eval runs. Real-adapter integration in v2 surfaces this.
2. **Rollback (on abort)**: when the recovery handler decides to abort (per ADR 0001 recovery loop), it walks the action log in REVERSE order:
   - For each entry with `reversible == True`: call `undo_action(entry)`. Log success/failure.
   - For each entry with `reversible == False`: log as "uncommitted side effect" — the world has been modified and we can't undo it. The final `AgentResponse` to the user must surface this honestly.

### Abort triggers

The recovery handler aborts (rather than retrying or clarifying) when:

- A tool fails with a non-retryable error (4xx user error, auth_error after refresh attempt)
- The model produces an `intent_error` mid-execution (e.g., contradictory subsequent steps)
- The wall-clock or step budget is exhausted (see ADR 0002 §6 timeouts)

### Execution (v2)

Real rollback execution is deferred to v2 because v1 uses fake adapters whose state lives in memory; rollback on a fake is just `state.pop(...)` and doesn't exercise the actually-hard part (idempotent undo against a real third-party API). v2's "real-adapter integration" milestone is when this ADR's execution semantics get tested in anger.

---

## Reasoning

[REASONING PROMPTS — expand each bullet into 1-2 sentences in your own voice. Documentation depth: brief, honest, readable in 6 months.]

- **WHY two patterns rather than one (verification gate for irreversible, undo for reversible)**: tools genuinely fall into two categories, and forcing one pattern to cover both is broken. You cannot undo a sent email; you cannot verify-then-delay every Notion write without leaking the mechanism into user experience. Two categories = two right patterns.
- **WHY the contract lands in v1 even though execution is deferred to v2**: adapter authors (Ronin in v2 when real adapters land) need to know what shape they're building against. A deferred contract = deferred adoption; declaring the contract now means v2 adapter work is mechanical rather than another design cycle.

- **WHY honesty over silent failure when irreversible side effects can't be undone**: partial success is a first-class case in real agent systems, not an edge case. If the agent has drafted an unsendable email but the Notion task already exists, the user has to know both facts. Hiding is worse than admitting — silent partial commits are the pathology of demo-only agents, and per the CV's "no fake AI agent wrapper" warning, that's the failure mode this project must not display.
- **WHY the recovery handler walks the action log in REVERSE order on abort**: correct undo order for compound actions. If step 1 creates a Notion task and step 2 assigns it to a user, the assignment must be undone before the task itself is deleted — same principle as reverse-order teardown in RAII / defer patterns.
- **WHY the `reversible: bool` flag lives on the adapter itself, not on the tool-call schema**: capabilities describe the ADAPTER's contract with the world, not the agent's request shape. An adapter author knows whether their tool can be undone; the agent shouldn't have to reason about it per-call.
- **WHY v1 fake adapters are all reversible**: makes the eval baseline clean — the verification-gate code path never triggers, so eval focuses on the core agent loop. When v2 introduces real irreversible adapters (Gmail send), the gate starts firing and adds a new eval dimension.
- **WHY reject two-phase commit as an alternative**: third-party APIs don't expose a prepare phase. You can't ask Notion to "prepare a task without creating it." Implementing prepare via a draft-workaround leaks the mechanism into the domain and breaks for tools where no draft concept exists. Real distributed transactions across heterogeneous APIs aren't solved in the literature for this case.

---

## Alternatives considered

### Alternative A — No rollback at all, bail on first error

When a tool fails, return an error response and leave the action log as it stands; user gets back "step 2 failed" with no cleanup.

why rejected — leaves the user in an inconsistent state with no recourse. Notion task exists, calendar event exists, but the user doesn't know about either because the agent stopped mid-flow. This is the "bad demo" failure mode. Per the CV's "no fake AI agent wrapper" warning — silent partial commits are the exact pathology of unserious agents.

### Alternative B — Two-phase commit (prepare/commit) across all tools

Every tool exposes a `prepare()` and a `commit()` method; the agent only fires `commit()` after every `prepare()` returns success.

why rejected — third-party APIs don't expose a prepare phase. You can't ask Notion to "prepare a task without creating it." Implementing prepare via a "draft" workaround (create with status=draft, then update to status=committed) leaks the mechanism into our domain and breaks for tools where no draft concept exists. Real distributed transactions across heterogeneous APIs are not solved in the literature for our case.

### Alternative C — Best-effort undo without contract, just retry the failed step harder

When a tool fails, don't roll back; instead retry with longer backoff and hope it resolves.

why rejected — for non-retryable errors (auth expiry, 4xx user error, irreversible side effect already committed downstream) retry does nothing. For genuinely transient errors, retry IS what the recovery loop does (per ADR 0001) — but at some point the budget is exhausted and we need a non-retry path. That non-retry path needs an undo contract.

### Alternative D — Per-tool ad-hoc rollback, no shared contract

Each tool adapter handles its own failure recovery in whatever way is natural; no shared `reversible` flag, no shared undo_action signature.

why rejected — the recovery handler in the agent core would need per-tool knowledge to decide what to do on abort. That moves orchestration logic into the wrong layer. Shared contract = orchestration stays in core, adapter just declares its capabilities.

---

## Consequences

### Positive

- Adapters that are honest about reversibility get the right runtime behaviour (rollback if reversible, verification gate if not)
- The user is never lied to about partial state — irreversible side effects are surfaced in the final response
- Contract is testable in v1 fakes; execution surface is small enough that v2 real-adapter authors have a clear target
- Adding a new adapter is mechanical: declare flag, implement execute + undo (or just execute if irreversible)

### Negative

- Every adapter author has to think about reversibility; adapters with subtle "kind of reversible" semantics (e.g., calendar event that's been seen by attendees) need careful design
- Verification gates increase user friction for irreversible operations — but the alternative is silent commits, which is worse
- v2 real-adapter integration must prove the undo path actually works against real APIs; this is the hard part

### Neutral

- Multi-phase tasks (draft → verify → send pattern for irreversible tools) become the natural follow-up — tracked as deferred per RONIN_SYNC.txt #14 / DEFERRED.md

---

## Open questions / follow-ups

- **Idempotent undo**: what if `undo_action()` itself fails partway? In v2, undo operations need their own retry / idempotency story. Sketch: undo is itself an action_log entry, so it can be retried.
- **"Partially reversible" tools**: e.g., calendar.create_event is technically reversible but if the event has been seen by other attendees, the undo isn't free of social cost. v1 treats this as a binary flag; v2 may need a richer "side effect class" taxonomy.

- **Multi-phase task pattern**: how does draft → verify → send compose with sub-task chains (deferred per RONIN_SYNC.txt #13)? Both deferred together to v2/v3.
- **Audit log of rollback decisions**: the recovery handler's choice to abort needs to be visible to Ronin's eval scorer; covered via the `response_type` enum in ADR 0002.

---

## References


- Two-phase commit (and why distributed transactions across heterogeneous APIs are hard): Gray & Lamport, "Consensus on Transaction Commit" (2006)
- Saga pattern (compensating transactions): Garcia-Molina & Salem, "Sagas" (1987) — closest classical analogue to what we're doing
- ADR 0001 for the recovery handler this ADR plugs into
- ADR 0002 for the response_type taxonomy
- RONIN_SYNC.txt #15 — decision locked
- DEFERRED.md — v2 trigger for execution semantics


---

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.